

Задача 1 (2 балла)

Рассмотрим код:

```
#include <iostream>

int main() {
    int x = 0;
    int y = 1;
    int z = 2;
    int t = (
        (z > 0 ? ++x : y++) = z++
    );
    std::cout << x << y << z << t;
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.
- 2) Ответьте на те же вопросы, если вместо `y++` написать `++y`.

Задача 2 (3 балла)

Рассмотрим программу:

```
#include <iostream>

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    int* p = a + 2;
    for (int* q = p - 2; q <= p + 2; ++q) {
        ++*q;
    }
    for (int i = -2; i <= 2; ++i) {
        std::cout << p[i];
    }
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.
- 2) Ответьте на те же вопросы, если вместо `int* p = a + 2;` написать `int* p = a[2];`
- 3) Ответьте на те же вопросы, если все оставить как было изначально, но в скобках после `for` вместо `++q` написать `++*q`.

Задача 3 (2 балла)

Рассмотрим программу:

```
int main() {
    int x = 0;
    int& y = x;
    const int& z = x;
    ++y;
    std::cout << x << y << z;
    int& t = z;
    int u = t;
    ++u;
    std::cout << x << y << z << t << u;
}
```

1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если вместо **int& t = z;** написать **int& t = y;**

Задача 4 (2 балла)

Рассмотрим программу:

```
#include <iostream>
int sum(int x, int y) {
    return x + y;
}
double sum(double x, double y) {
    return x + y + 1;
}
int main() {
    int c = sum(1, 2);
    double d = sum(3, 4);
    std::cout << c << ' ' << d;
}
```

1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.

2) Ответьте на те же вопросы, если бы первая функция принимала оба аргумента не int, а float, и возвращаемый тип тоже имела бы не int, а float.

Задача 5 (3 балла)

Рассмотрим программу:

```
#include <iostream>

class A {
private:
    int* p = new int;
public:
    A(int x) {
        p = new int(x);
    }
    ~A() {
        delete p;
    }
    int& getInt() {
        return *p;
    }
    A(const A&) = default;
};

int main() {
    A a(5);
    A b = a;
    std::cout << a.getInt() << b.getInt();
}
```

- 1) Какая ошибка возникнет, если запустить эту программу, и почему? Дайте развернутое объяснение.
- 2) Не меняя остальной код, реализуйте конструктор копирования для данного класса таким образом, чтобы код запускался и завершался успешно.
- 3) Какая еще проблема, кроме только что исправленной, есть в данном коде? Как нужно исправить конструктор класса A, чтобы устранить эту проблему?

Задача 6 (2 балла)

Рассмотрим программу:

```
#include <iostream>
int x = 0;

struct S {
    int* p = new int(0);
    int& a = x;

    void f() /*const*/ {
        *p = 1;
        a = 2;
        std::cout << *p;
    }
    ~S() { delete p; }
};
```

```
int main() {
    const S s;
    s.f();
    std::cout << x;
}
```

- 1) Вызовет ли этот код CE, RE, UB или все отработает корректно? В первом случае объясните, какая именно случится ошибка (или что вызовет UB) и почему, во втором -- ответьте, что выведется на экран и почему.
- 2) Ответьте на те же вопросы, если раскомментировать слово `const`.

Задача 7 (2 балла)

Рассмотрим код:

```
struct UserId {
    int id = 0;

    /*explicit*/ UserId(int id): id(id) {} // 1

    /*explicit*/ operator int() { // 2
        return id;
    }
};

int operator+(UserId a, UserId b) {
    return a.id + b.id;
}

int main() {
    UserId id = UserId(5);
    std::cout << id + 1;
}
```

- 1) Из-за чего в этом коде возникает ошибка компиляции? Объясните развернуто.
- 2) Что изменится, если раскомментировать слово `explicit` в первом случае? А если раскомментировать оба слова `explicit`? В каждом случае объясните ответ.

Задача 8 (2 балла)

Рассмотрим программу:

```
#include <iostream>

struct Granny {
    int x = 0;
    void f() {}
};

struct Mom: private Granny {
    //friend struct Son;
    int y = 1;
protected:
    void g() {
        std::cout << x;
        f();
    }
};

struct Son: public Mom {
    int z = 2;
    void h() {
        std::cout << x << y;
        g();
    }
};

int main() {
    Son s;
}
```

- 1) Скомпилируется ли эта программа? Если да, то просто напишите “да”, а если нет, то укажите все места, в которых возникнут ошибки компиляции, и почему они возникнут.
- 2) Что изменится, если раскомментировать строчку `friend struct Son;`? Вновь объясните ответ.

Задача 9 (2 балла)

Рассмотрим программу:

```
#include <iostream>
struct Base {
    void f() { std::cout << 1; }
};
struct Derived: public Base {
    void f() /* override */ { std::cout << 2; }
};
int main() {
    Derived d;
    d.f();
    static_cast<Base&>(d).f();
}
```

- 1) Скомпилируется ли она (если нет, то почему)? Если да, то что можно утверждать про результат вывода на экран? Объясните свои ответы.
- 2) Что изменится, если раскомментировать слово `override`? Вновь объясните ответ.

Задача 10 (2 балла)

Рассмотрим код:

```
#include <iostream>

struct Granny {
    /*virtual*/ void foo() { std::cout << "Granny\n"; }
};

struct Mother: public Granny {
    void foo() { std::cout << "Mother\n"; }
};

struct Father: public Granny {
    void foo() { std::cout << "Father\n"; }
};

struct Son: public Mother, public Father {};

int main() {
    Son s;
    Mother& m = s;
    dynamic_cast<Father&>(m).foo(); // 1
    dynamic_cast<Son&>(m).foo(); // 2
}
```

Оказалось, что данный код не компилируется: ошибки компиляции возникают в обеих выделенных строках. Ответьте на вопросы:

- 1) Почему возникают ошибки компиляции в данном коде?
- 2) Поменяется ли что-нибудь, если раскомментировать слово `virtual`? Исчезнут ли ошибки, появятся ли другие вместо них? Поясните подробно.

Задача 11 (2 балла)

Рассмотрим следующий код:

```
#include <iostream>
template<int N>
struct Sum {
    static const int value = N + Sum<N-1>::value;
};

int main() {
    std::cout << Sum<100>::value;
}
```

- 1) Какая ошибка произойдет при попытке его скомпилировать и запустить? Объясните причину этой ошибки.
- 2) Допишите нужный код, чтобы эта программа успешно компилировалась и выполнялась без ошибок, а на экран бы выводилось значение суммы натуральных чисел от 1 до 100.

Задача 12 (2 балла)

Рассмотрим программу:

```
#include <iostream>

template <typename T>
struct S {
    T x;
    S(T x): x(x) {}
};

int main() {
    S<int> s1(5);
    S<double> s2 = s1;
    std::cout << s1.x << s2.x;
}
```

- 1) Какая ошибка произойдет при попытке ее скомпилировать и запустить? Объясните, почему она произойдет.
- 2) Не меняя код функции main, допишите что-нибудь в тело структуры S, так, чтобы код успешно компилировался и выполнялся без ошибок.